# INF1103: Programming Fundamentals with DevOps

## Week 10: Object Oriented Relationships Part 2

***This session is starting shortly...***

***While you wait, go to XSITE and download Animals.zip and get it running, at least ready to run***
***The code as it is will throw an error, don't worry!***

***It is in the Lectures folder!***

# Some points from last week

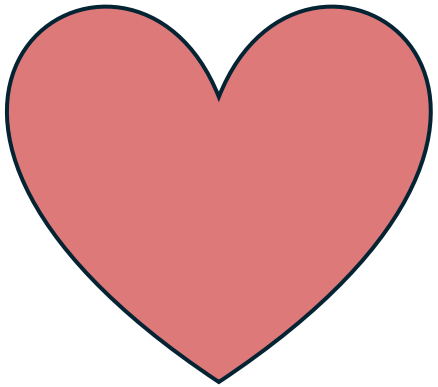
- Week 8 lab due on Tuesday (tomorrow) 23:59
- Week 9 ++ labs due on Friday 23:59
- One class per file (don’t forget!)

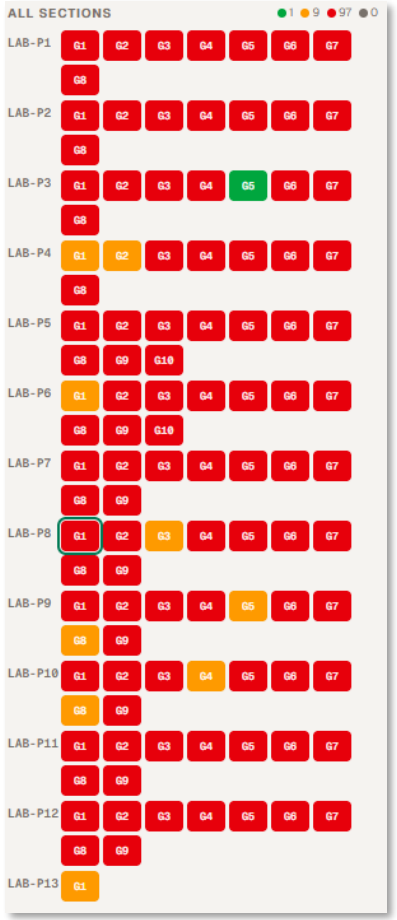
- Task Management in TeamAId for project is important

Lab P3

G5

***AWESOME!***








# Some points for this week

- Exploring more interactive sessions (vs everyone back on campus)
- Will ask you questions throughout, get you to think, test your understanding
- We will get you to talk to each other (in your groups) on TeamAId
- Talk to the AI on TeamAId
- Reflect on how that all went!

# Thought Process

- **Predict:** Form your own expectation before seeing the answer
- **Commit and justify:** A prediction without a reason is a guess, you should be able to justify why you went a certain direction
- **Check:** See the slide, run the code, or look at the AI output.
- **Reconcile:** If the check didn’t match, work out why. Which assumption was wrong? What line gave it away? This is the step most often skipped
- **Transfer:** Use the corrected idea somewhere new, like in your lab (we won’t do robots!) or in your project (you won’t do product?)

# Learning Outcomes

- Interaction and Interoperation
- Association
- Aggregation
- Composition
- Polymorphism

# Interaction and Interoperation

- What is this?



Do not edit
How to change the design



# What objects are needed to build the chair?

The Slido app must be installed on every computer you're presenting from

slido

Do not edit
How to change the design



**Choose one part of the chair and something that depends on it and why that 'thing' depends on that part**

ⓘ The Slido app must be installed on every computer you're presenting from

slido

# Interaction and Interoperation

- Class **Interaction** is how classes **communicate and work with each other** in Object Oriented Programming
- Class **Interoperation** is how classes **work together and cooperate** to achieve a **larger task**
- Interaction is communication between classes, interoperation is how they work together as a ***system*** (to achieve a larger task)



# Interaction and Interoperation

- How classes interact and interoperation determine how their relationships are implemented in code
- Classes can have tightly coupled or loosely coupled relationships
  - **Tightly coupled :** classes depend heavily on each other, so changes to one class may require changes to another
  - **Loosely coupled:** Classes have fewer dependencies with each other, making them easier to modify test and reuse independently.
- Generally, reusable and maintainable code benefits from loose coupling although some situations benefit from tight coupling
- Loosely coupled classes are ***easier*** to modify, test and reuse independently because they depend less on the internal implementation of other classes

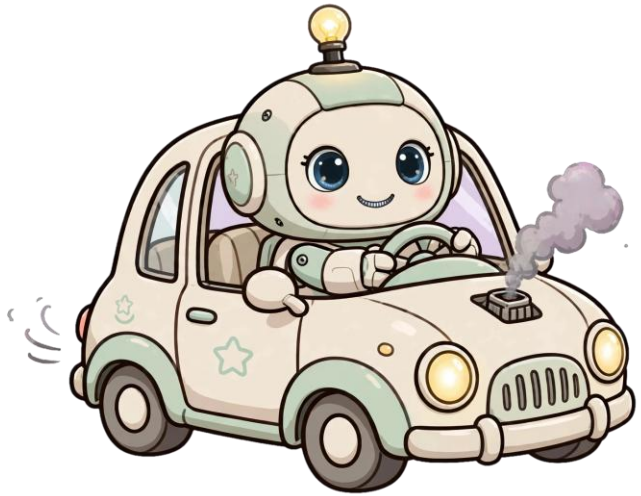
# Interaction and Interoperation

- This relationship between how classes are linked forms the basis for the relationships of:
  - Association
  - Aggregation
  - Composition

- And help us understand how each one is differentiated from the other.

# Guiding Question

- What happens to the part when the whole goes away?



Do not edit
How to change the design



**If the car is a whole single object (don't think about the system) what other objects might interact with it?**

ⓘ The Slido app must be installed on every computer you're presenting from

slido

Do not edit
How to change the design

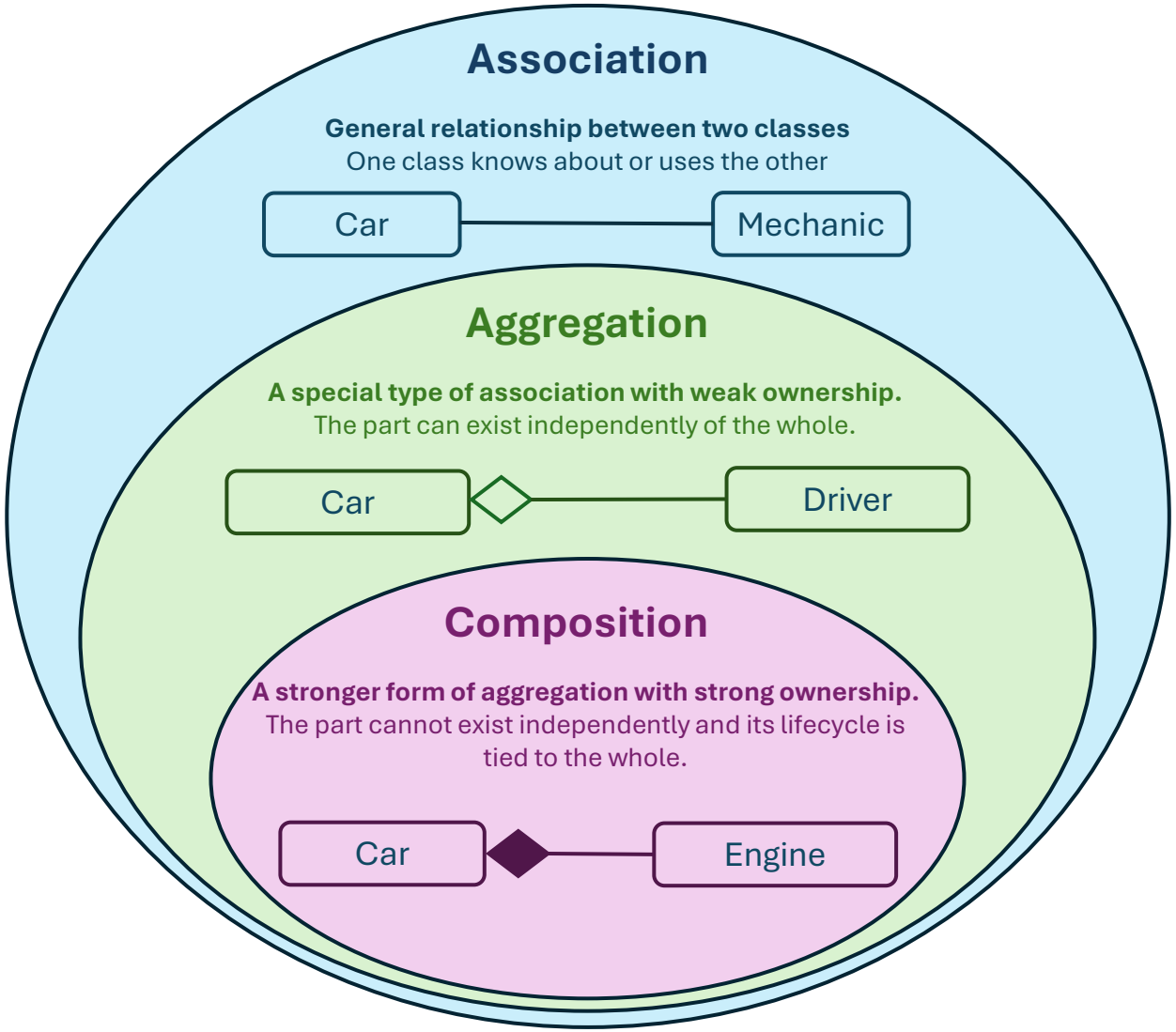


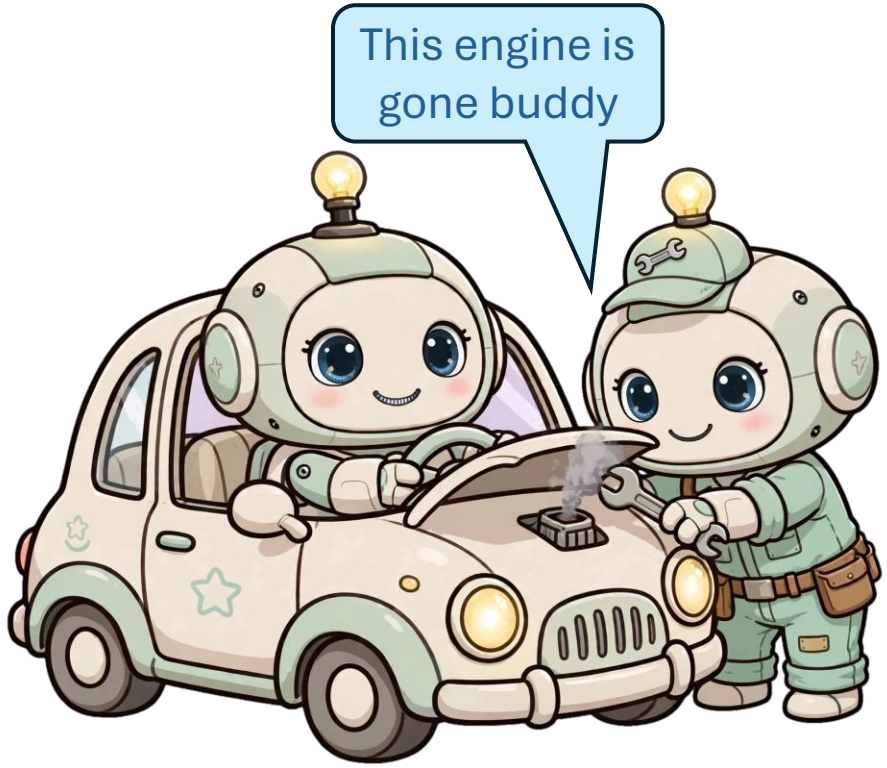
# If the car is made of parts, what objects does it need that it can not work without?

ⓘ The Slido app must be installed on every computer you're presenting from

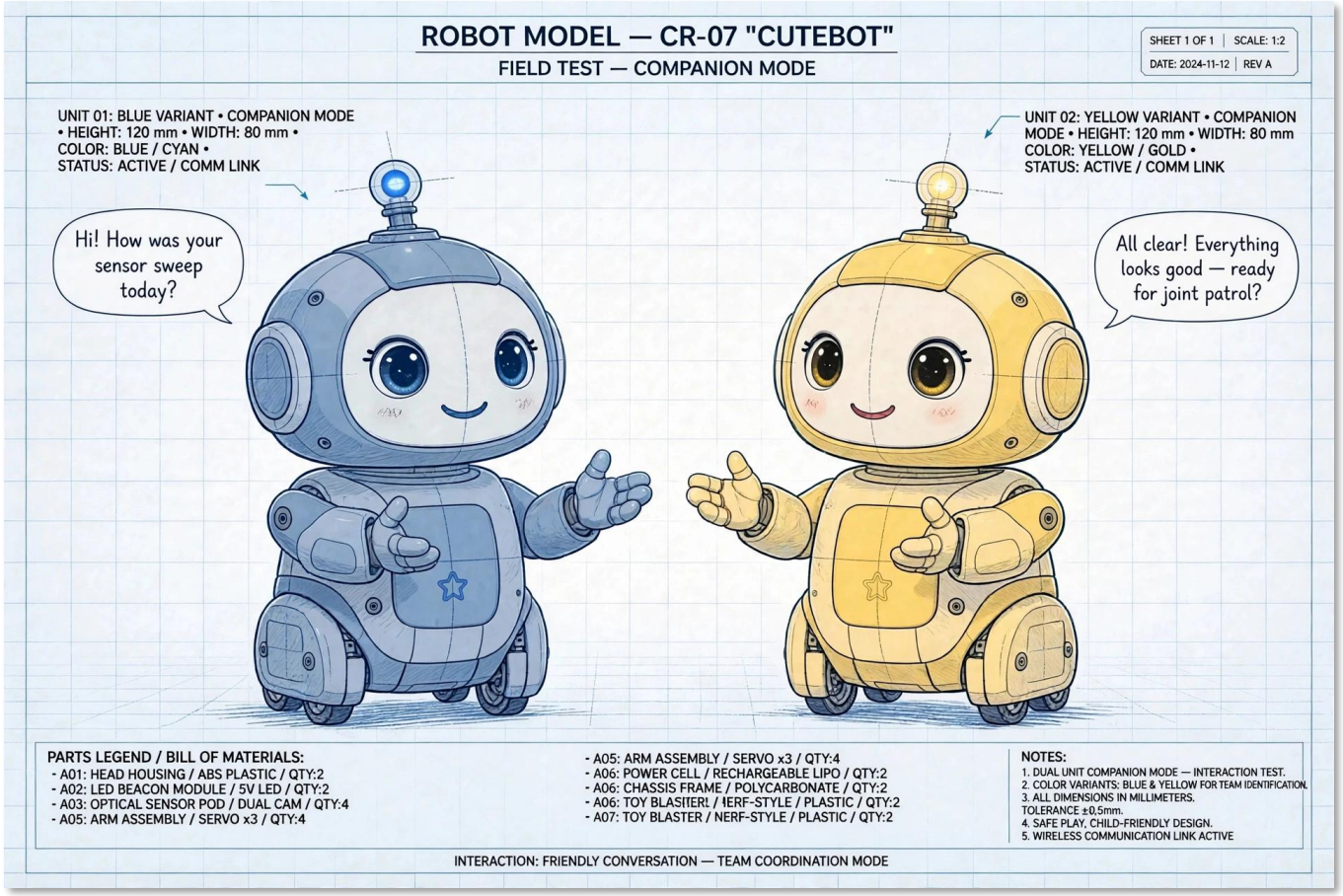
slido

# Association / Aggregation / Composition

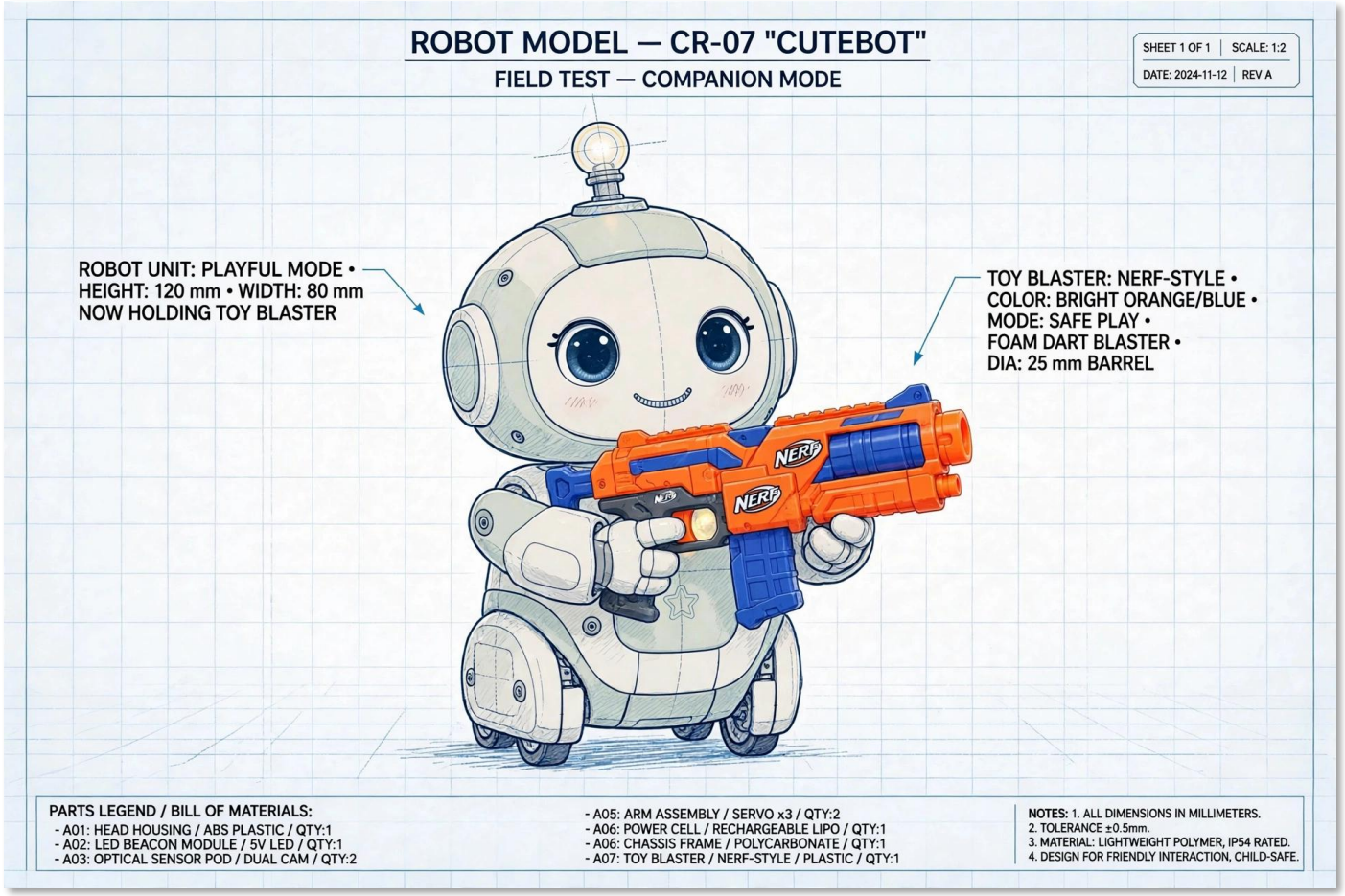


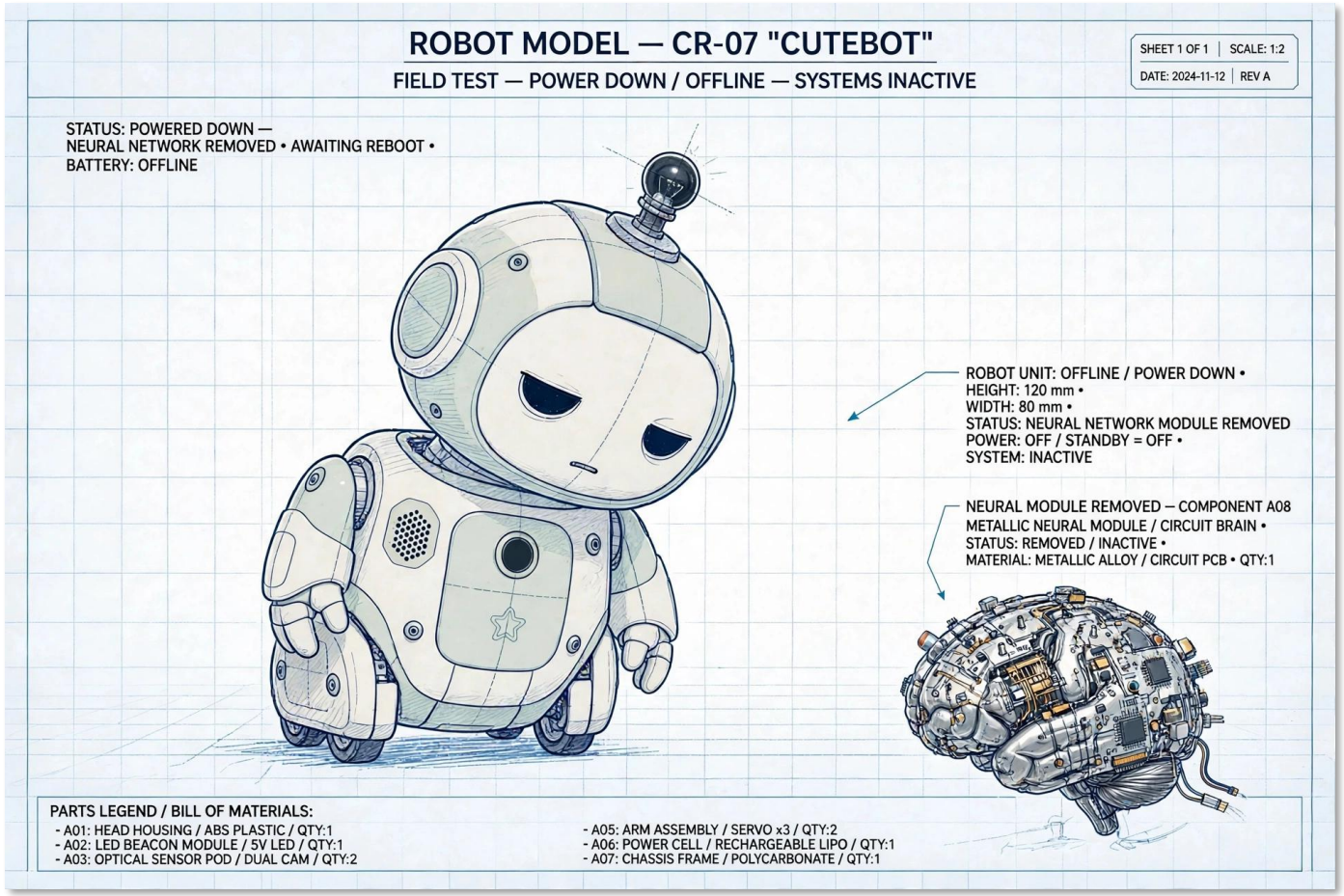
# What do you see? (Association / Aggregation / Composition)



# What do you see? (Association / Aggregation / Composition)



# What do you see? (Association / Aggregation / Composition)



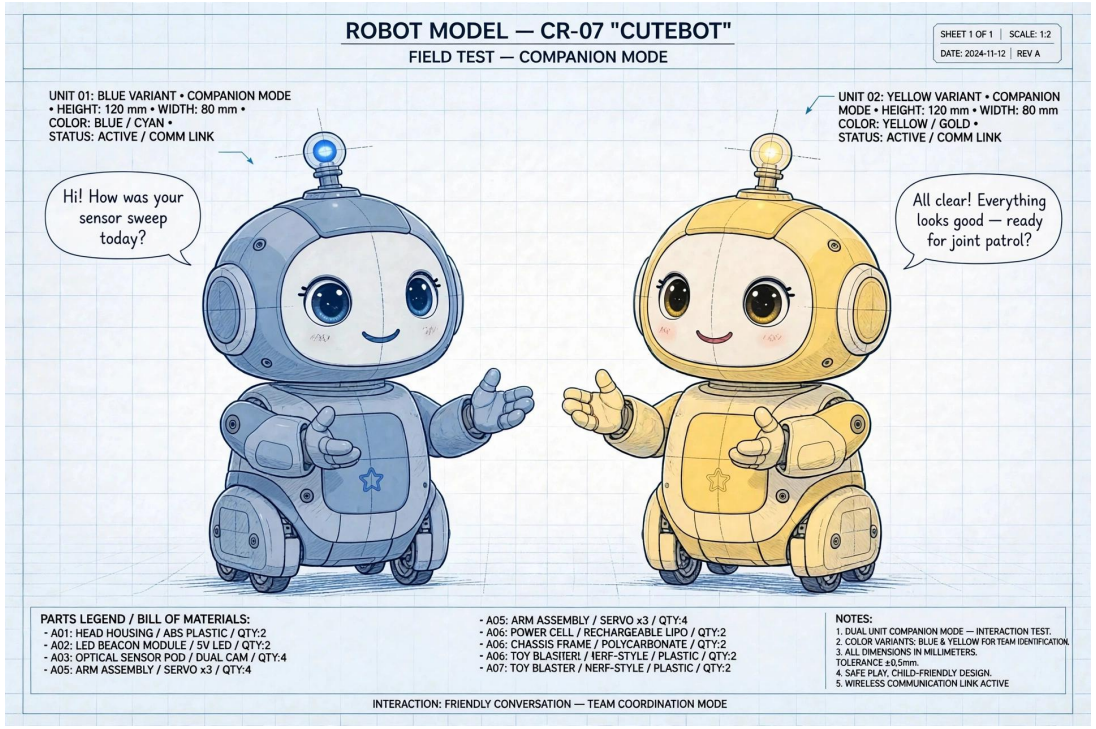
# Association / Aggregation / Composition

- Lets go through each of these class relationships in more detail

- I will show you sample code but some things to look at:
  - You should be able to see the relationship in the code
  - You should be able to predict what the code looks like once we have decided on the relationship
  - UML Class diagrams often have and attribute or method that also sign posts the relationship
  - You should have decided all of this well before you wrote the code!

# Association

- Association (uses relationship)
  - General relationship between classes
  - No ownership implied
  - They could work together, ask each other to help them do work, but no ownership
  - Can be unidirectional or bidirectional
  - In simple terms, they can talk to each other but they don’t own one or the other




***Do the robots need to know about each other?***

Do not edit
How to change the design



**Where would you expect to see the robot association relationship stored in code? (how would one robot know about the other?)**

ⓘ The Slido app must be installed on every computer you're presenting from

slido

# Association

```
class Robot:
    def __init__(self, name):
        self.name = name

    def talk_to(self, other_robot):
        print(f"{self.name} talks to {other_robot.name}")
```

***Note:***

- ***In this case robots don't need a ref to each other.***
- ***However, association is possible with each having a reference to each other.***
- ***This is a design decision but you must understand the relationship you intend***
- ***No ownership, just awareness***

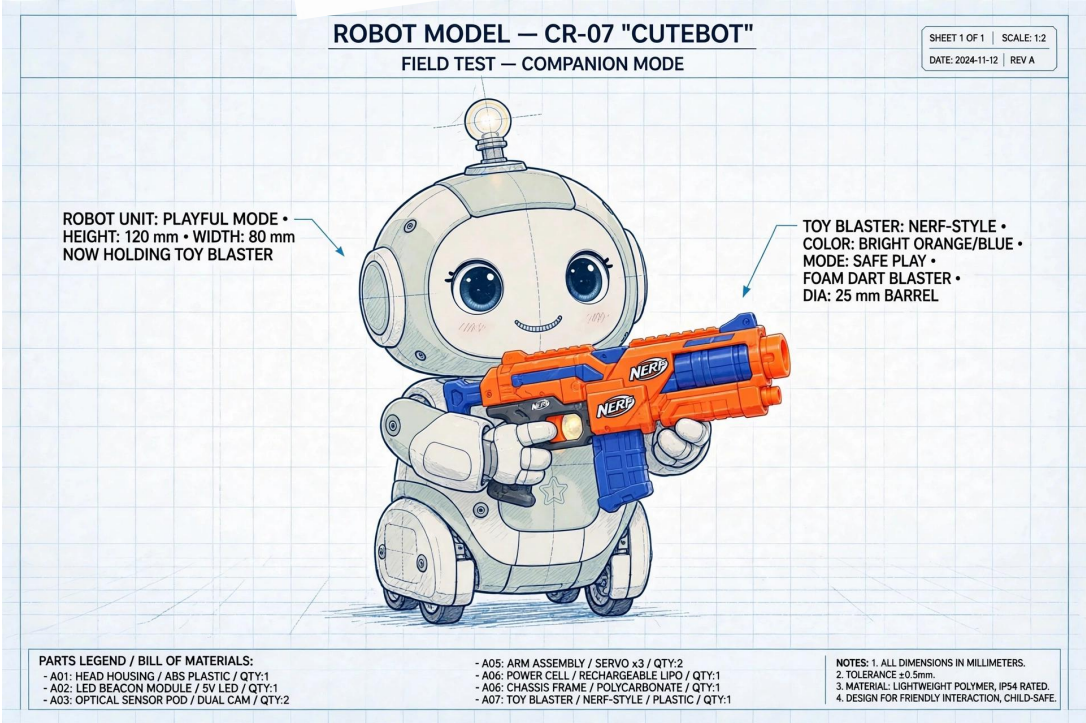
```
robot1 = Robot("Robo")
robot2 = Robot("Buddy")

robot1.talk_to(robot2)
robot2.talk_to(robot1)
```

# Aggregation

- A type of association with weak ownership (has a relationship)
- The part can exist independently of the whole
- Often used for “has-a” relationships where the part is shared or reusable




***Does the Nerf-Gun need to know about the Robot, does the Robot need to know about the Nerf-Gun***

Do not edit
How to change the design



**Where would you expect to see the Nerf-Gun aggregation relationship stored in the robots code? (how would the robot know about the gun?)**

ⓘ The Slido app must be installed on every computer you're presenting from

slido

# Aggregation

```
class NerfGun:
    def fire(self):
        print("Nerf dart fired!")
```

```
class Robot:
    def __init__(self, name, gun):
        self.name = name
        self.gun = gun

    def fire_gun(self):
        self.gun.fire()
```

```
# Create the gun independently
gun = NerfGun()

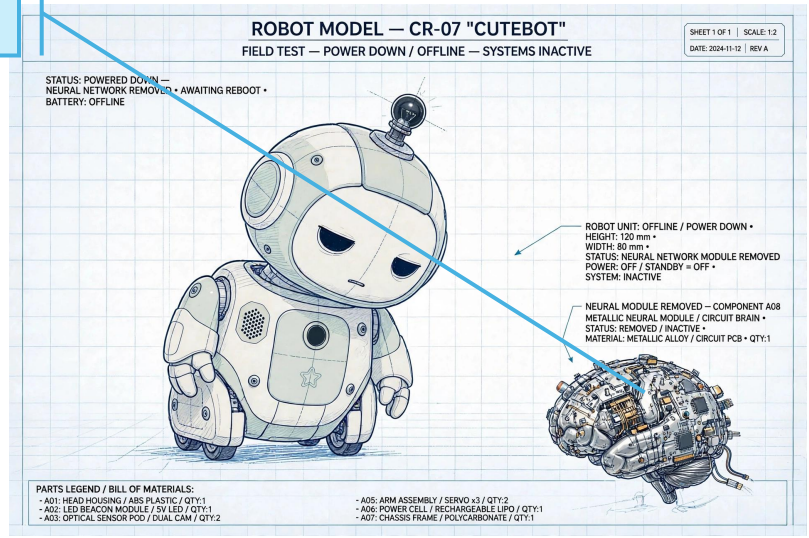
# Give the existing gun to a robot
robot = Robot("Robo", gun)

robot.fire_gun()
```

# Composition

- A stronger form of aggregation with strong ownership (owns relationship)
- This is considered a terminal relationship, if the owner dies, all of the composition parts die too
- The part cannot exist independently
- The lifecycle of the part is tired to the whole

Composition




Aggregation



Do not edit
How to change the design



**Where would you expect to see the robot composition relationship stored in code? (how would the robot talk to the brain)**

ⓘ The Slido app must be installed on every computer you're presenting from

slido

# Composition

```
class Brain:
    def think(self):
        print("Thinking...")


class Robot:
    def __init__(self, name):
        self.name = name
        self.brain = Brain()

    def think(self):
        self.brain.think()


robot = Robot("Robo")
robot.think()
```

# For Fun

```
class Robot:
    def __init__(self, name):
        self.name = name
        self.gun = None

    def pick_up(self, gun):
        if self.gun is not None:
            print(f"{self.name} is already holding a gun!")
            return

        self.gun = gun
        print(f"{self.name} picks up the gun.")

    def drop(self):
        if self.gun is None:
            print(f"{self.name} isn't holding a gun.")
            return None

        gun = self.gun
        self.gun = None
        print(f"{self.name} drops the gun.")
        return gun
```

```
class NerfGun:
    def fire(self):
        print("Nerf dart fired!")
```

```
# Robot B can now use it
robotB.gun.fire()

# Create two robots and one independent gun
robotA = Robot("Robot A")
robotB = Robot("Robot B")
gun = NerfGun()

# Robot A picks up the gun
robotA.pick_up(gun)

# Robot A drops it, making the gun available
dropped_gun = robotA.drop()

# Robot B picks up the same gun
robotB.pick_up(dropped_gun)
```

Do not edit
How to change the design



# From a coding perspective, should our Robot be able to join the animal family

ⓘ The Slido app must be installed on every computer you're presenting from

slido

Do not edit
How to change the design



# If you answered Yes, why?  If you answered No, Why? (Format say Yes: Because, or No: Because)

ⓘ The Slido app must be installed on every computer you're presenting from

slido

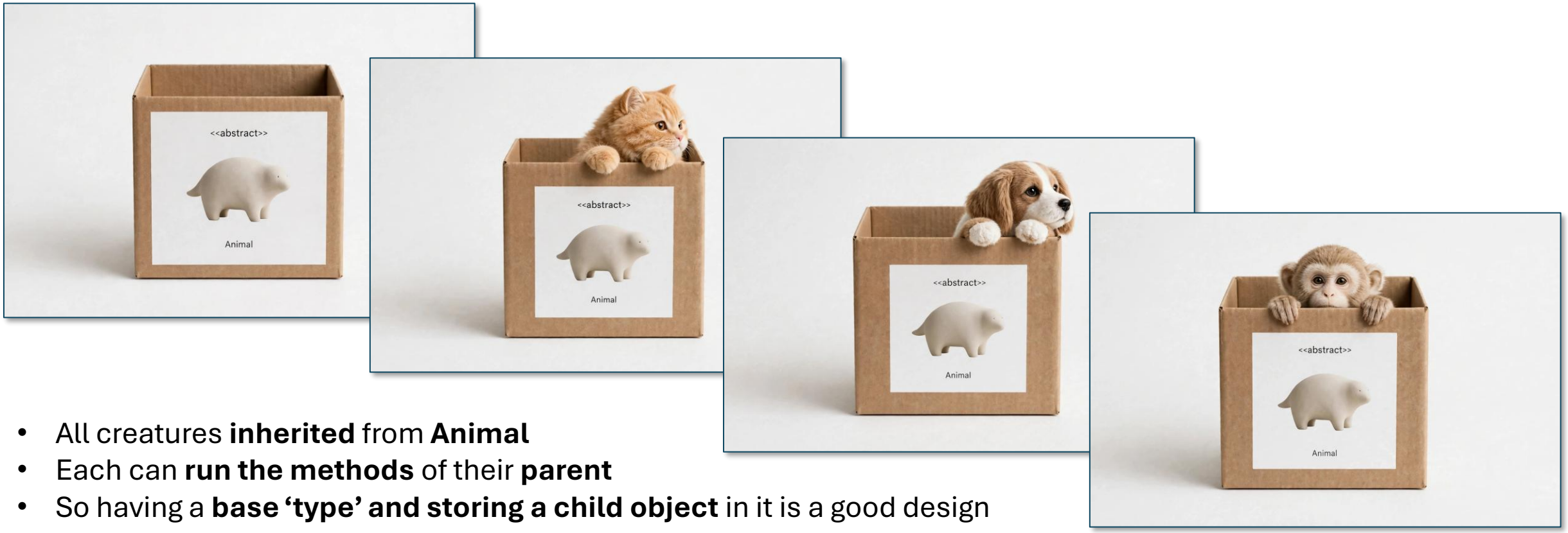
# Polymorphism

- **Polymorphism** allows **different types of objects** or functions to be **handled through a single, common *interface***

- The word comes from the Greek ‘Poly’ meaning **many** and ‘morph’ meaning **forms**

- **Run-time Polymorphism (Method Overriding):** A child class provides its own specific implementation of a method already defined in its parent/base class. When you call the method through the base class reference, the program automatically runs the child class's version.

# Polymorphism




- All creatures **inherited** from **Animal**
- Each can **run the methods** of their **parent**
- So having a **base 'type' and storing a child object** in it is a good design decision
- When we call **Speak() from a cat, dog or monkey,** since the parent has Speak() we know that the **code will work**, children can **override Speak()** from the parent, this 'animal' that behaves different depending on what is in the box based on overriding, this is polymorphism

# Polymorphism

- Other languages do this explicitly with type (Java example)
  - Animal animal1 = new Dog()
  - Animal animal2 = new Cat()

- The Animal class has a speak method() the Cat overrides with ‘Meow’ and the dog with ‘Woof’

- Animal1.speak() # ‘Woof’ is the output
- Animal2.speak() # ‘Meow’ is the output

# Polymorphism

- A robot is NOT an animal
- It shouldn’t be in the same variable as a cat or a dog
- Animal animal3 = new Robot() # code error in Java
- What about Python?
  - Good / Bad / Why?



# Polymorphism

- Python hides types and how it implements things from us a bit!
  - Dynamic Typing and Duck Typing

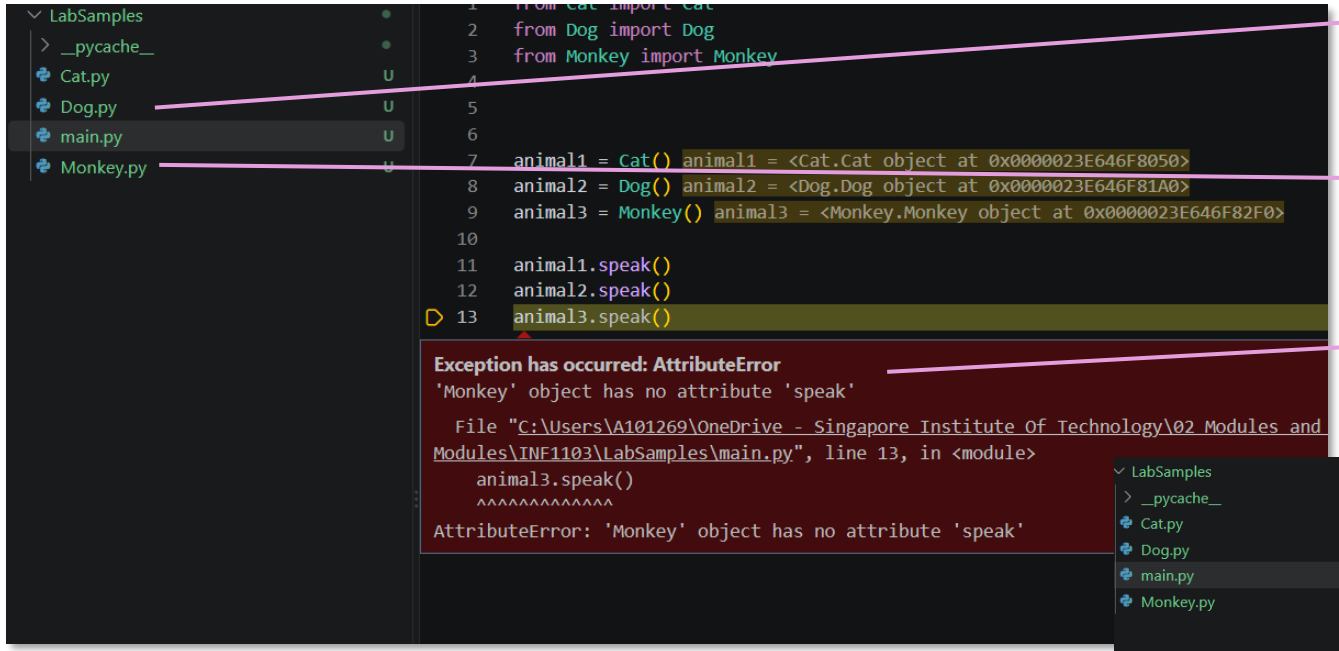
- animal = dog()
- animal = cat()
- animal = robot()

- animal variable starts as a dog but ends as a robot

- Python is ok with this, other languages are not

# Polymorphism

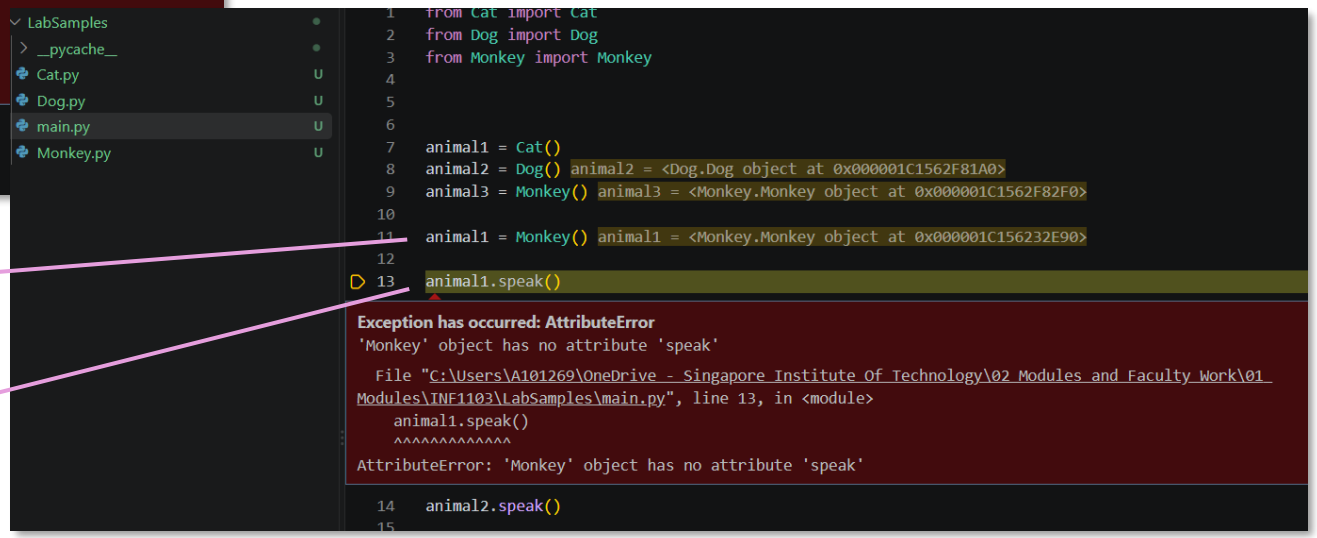
- So no harm lah, let python do its thing right? (wrong):




No Inheritance or Interfaces

No Speak() Method but nothing compile time to catch it

Runtime Error # BAD




Animal1 type changes during runtime

Would work for Cat, but not for Monkey

# Polymorphism

- ***FIX THE PROBLEM***
  - Download some broken code from XSITE
  - Discuss on TeamAId group chat how to fix it
  - Ask your TeamAId AI Teammate to suggest a fix
    - What instructions did you give it before asking for a fix?
    - What conversation did you have with it after it suggested a fix?
    - At what point did you decide that it had a good enough answer for you?

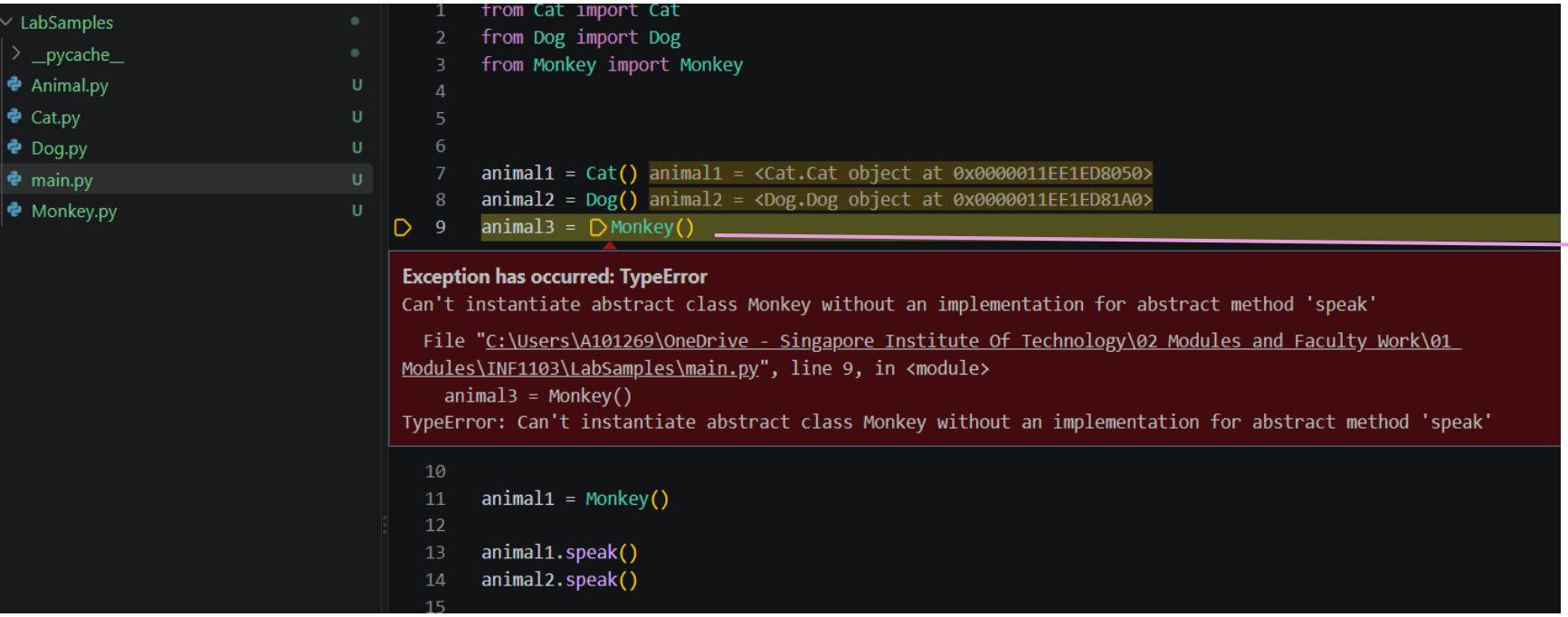


https://teamaid-sit.vinova.sg/

Access to TeamAId

# Polymorphism

- The fix, abstract base class animal and enforce speak from an abstract method in animal, then cat, dog, monkey all inherit
- If monkey doesn't implement speak, still runtime issue but caught earlier
  - Other languages it wouldn't even compile with such issues



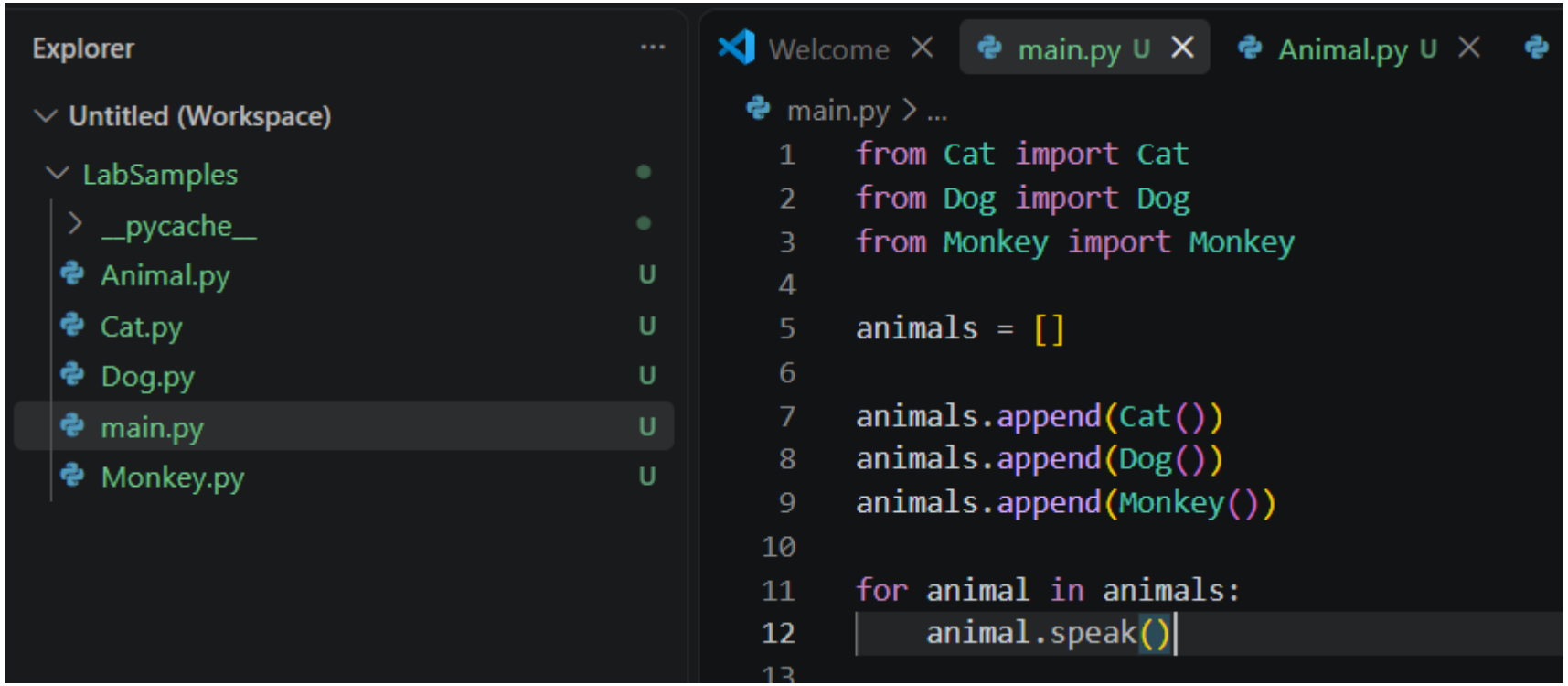

Runtime Error but caught early and tells us what is missing

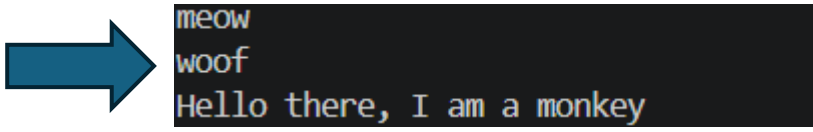
# Polymorphism

- Abstract Inherited Classes establish promised methods for children to implement
- When we call a child method we can ensure that method exists
- Interfaces offer similar functionality: promised methods but functional ‘must be able to do’ vs ‘is-a’ relationship
- Interfaces are also good for polymorphism and in other languages can also we used as a type to achieve the same goal

# Polymorphism

- This is awesome because:



# Polymorphism

- Even better:

```
from Dog import Dog
from Cat import Cat
from Monkey import Monkey
from AnimalManager import AnimalManager

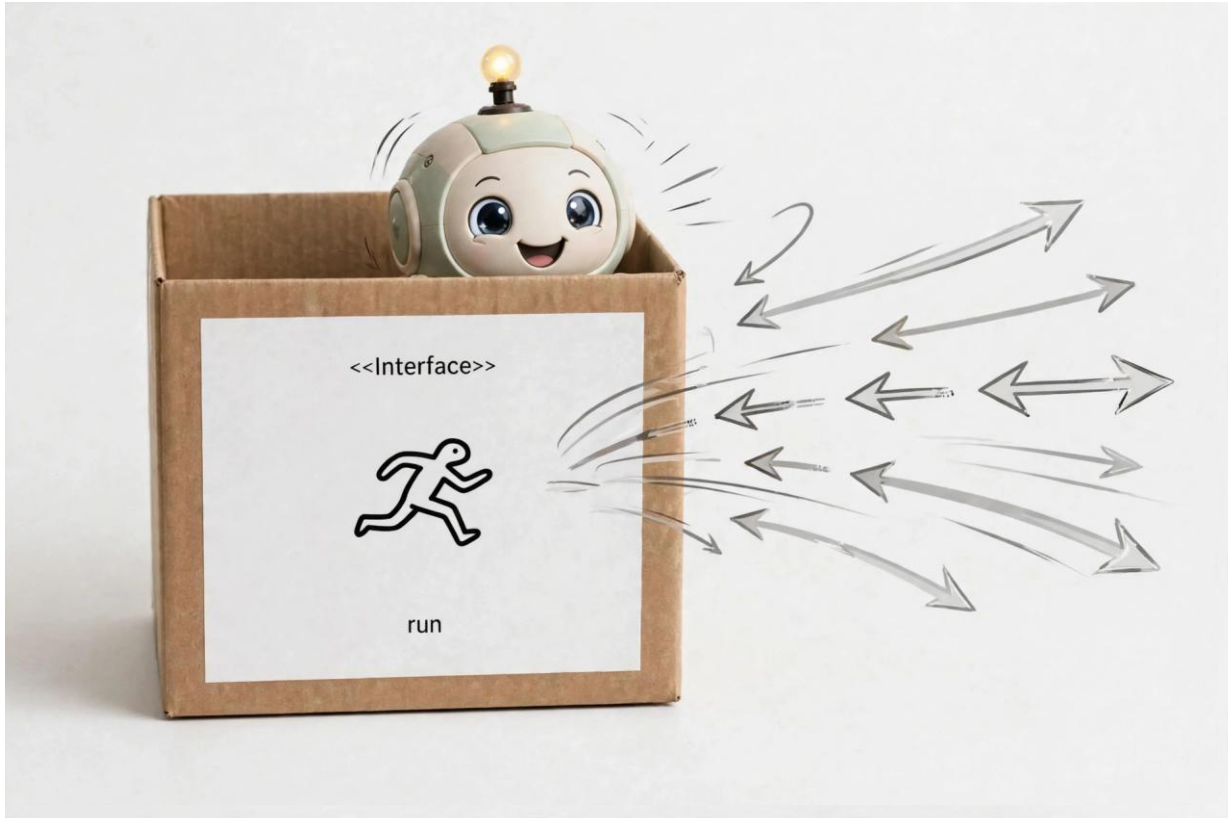

animal1 = Dog()
animal2 = Cat()
animal3 = Monkey()


animalManager = AnimalManager()
animalManager.register_animal(animal1)
animalManager.register_animal(animal2)
animalManager.register_animal(animal3)


animalManager.make_animals_speak()
```

# Polymorphism

- So, can our robot friend still run with the animals
- Yes! Interfaces, in python using same setup as abstract classes but with a ‘must-be-able-to’ relationship (design decision)
- Reminder: Other languages will have a keyword or separate way to establish interfaces (but their purpose is the same)



# Reflection!

- INF1103 Week 10 Reflection – Fill out form

- Be positive! Think about how this session, your team and AI supported your learning today!

# Thanks!!!

